

INSTITUTO SUPERIOR DE ENGENHARIA DO PORTO

Pós-Graduação em Sistemas Computacionais Embebidos
Sistemas Operativos de Tempo-Real (PGSCE-SIOPTR)

SIOPTR: TT7: Week 4

My Own KERnel (MOKER) version: Framework System Calls

Paulo Baltarejo Sousa
pbs@isep.ipp.pt
2025/26

1 Introduction

The goal of this feature is to enable and disable tracing mechanism. For that, we have to create a new system call.

In order to implement a new system call to enable or disable the `MOKER_TRACING` mechanism it is required a set of steps.

2 Enabling and disabling MOKER_TRACING mechanism

The main idea is to add a start/stop mechanism for collecting data. For that purpose, we change the implementation `MOKER_TRACING` mechanism, `trace.c`, to support such a mechanism. This mechanism is based on a simple variable, called `enabled`.

```
...
#include "trace.h"

struct trace_evt_buffer trace;

//0 - means disabled
//!0 - means enable
unsigned int enabled = 0;

...
void moker_trace(enum evt event, int number, struct task_struct *p){
    unsigned long long time;
    if(enabled){
        time = ktime_to_ns(ktime_get());
        enqueue(event, time, number, p);
    }
}

void enable_tracing (unsigned int e){
    enabled = e;
}
```

We also need to update `trace.h`.

```
#ifndef __TRACE_H_
#define __TRACE_H_
...
void moker_trace(enum evt event, int number, struct task_struct *p);
void enable_tracing (unsigned int e);
#endif
```

3 Create a System call

Create a new file called `syscalls.h` file in the `linux-6.19-moker/kernel/moker` directory. This file will host all MOKER-related system call prototypes.

Write into it, the prototype of the new system call called `sys_moker_tracing` that receive a parameter to `enable` (`enable` equal to 1 or 0, for enabling or disabling the TRACING mechanism, respectively).

4 Implementing the System call

Create two new files called `syscalls.c` and `syscalls.h` into the `linux-6.19-moker/kernel/moker` directory

These two file will host all MOKER-related system call implementations. The content of `syscalls.h` file is:

```
#ifndef __SYSCALLS_H
#define __SYSCALLS_H
int do_moker_tracing (unsigned int enable);
#endif
```

The content of `syscalls.c` file is:

```
#include <linux/syscalls.h>
#include "trace.h"
SYSCALL_DEFINE1(moker_tracing, unsigned int, enable)
{
    printk("MOKER: moker_tracing:[%d][%d]\n", (int) enable, current->pid);
    return do_moker_tracing(enable);
}

int do_moker_tracing (unsigned int enable){
    #ifdef CONFIG_MOKER_TRACING
        printk("MOKER: sys_moker_tracing:[%d][%d]\n", (int) enable, current->pid);
        trace_enable(enable);
    #endif
    return 0;
}
```

Include the system call function in the Linux kernel compilation process, adding `obj-y += syscalls.o` to the `linux-6.19-moker/kernel/moker/Makefile` file.

```
# MOKER framework makefile

obj-$(CONFIG_MOKER_TRACING) += trace.o

obj-y += syscalls.o
```

5 Getting a System Call Number

Add a new entry to the system call table. This is located at `linux-6.19-moker/arch/x86/entry/syscalls/syscall_64.tbl`.

The number of the new system call is 436. This system call is used for x86 and x86_64 systems, so the ABI is `common`. The system call name is `moker_tracing` and the entry point is `sys_moker_tracing`.

```
# SPDX-License-Identifier: GPL-2.0 WITH Linux-syscall-note
#
# 64-bit system call numbers and entry vectors
#
# The format is:
# <number> <abi> <name> <entry point> [<compat entry point> [noreturn]]
#
# The __x64_sys_*() stubs are created on-the-fly for sys_*() system calls
#
# The abi is "common", "64" or "x32" for this file.
#
0 common read    sys_read
1 common write   sys_write
2 common open    sys_open
3 common close   sys_close
4 common stat    sys_newstat
5 common fstat   sys_newfstat
6 common lstat   sys_newlstat
...
463 common setxattratrat sys_setxattratrat
464 common getxattratrat sys_getxattratrat
465 common listxattratrat sys_listxattratrat
466 common removexattratrat sys_removexattratrat
467 common open_tree_attr sys_open_tree_attr
468 common file_getattr sys_file_getattr
469 common file_setattr sys_file_setattr
470 common listns sys_listns
#ifdef CONFIG_MOKER_FRAMEWORK
471 common moker_tracing sys_moker_tracing
#endif
...
```

6 Putting MOKER System Calls Available

Include the `linux-6.19-moker/kernel/moker/syscalls.h` file in the `linux-6.19-moker/include/linux/syscalls.h` file.

```

/* SPDX-License-Identifier: GPL-2.0-only */
/*
 * syscalls.h - Linux syscall interfaces (non-arch-specific)
 *
 * Copyright (c) 2004 Randy Dunlap
 * Copyright (c) 2004 Open Source Development Labs
 */

#ifndef _LINUX_SYSCALLS_H
#define _LINUX_SYSCALLS_H

...
/* for __ARCH_WANT_SYS_IPC */
long ksys_semtimedop(int semid, struct sembuf __user *tsops,
unsigned int nsops,
const struct __kernel_timespec __user *timeout);
long ksys_semget(key_t key, int nsems, int semflg);
long ksys_old_semctl(int semid, int semnum, int cmd, unsigned long arg);
long ksys_msgget(key_t key, int msgflg);
long ksys_old_msgctl(int msqid, int cmd, struct msqid_ds __user *buf);
long ksys_msgrcv(int msqid, struct msgbuf __user *msgp, size_t msgsz,
long msgtyp, int msgflg);
long ksys_msgsnd(int msqid, struct msgbuf __user *msgp, size_t msgsz,
int msgflg);
long ksys_shmget(key_t key, size_t size, int shmflg);
long ksys_shmdt(char __user *shmaddr);
long ksys_old_shmctl(int shmid, int cmd, struct shmid_ds __user *buf);
long compat_ksys_semtimedop(int semid, struct sembuf __user *tsems,
unsigned int nsops,
const struct old_timespec32 __user *timeout);
long __do_semtimedop(int semid, struct sembuf *tsems, unsigned int nsops,
const struct timespec64 *timeout,
struct ipc_namespace *ns);

int __sys_getsockopt(int fd, int level, int optname, char __user *optval,
int __user *optlen);
int __sys_setsockopt(int fd, int level, int optname, char __user *optval,
int optlen);

#ifdef CONFIG_MOKER_FRAMEWORK
#include "../../kernel/moker/syscalls.h"
#endif

#endif

```

7 Compiling

1. Compile and install the kernel:

```

sudo ./kcompile.sh
sudo reboot

```

8 Getting data

After rebooting and logging in.

1. Check if the system is tracing data:

```
cat /proc/moker_trace
```

The system is not tracing, because by default the tracing mechanism is disabled.

	C (ms)	T (ms)	D (ms)	Prio
τ_1	2	6	5	
τ_2	3	12	12	
τ_3	4	15	15	

2. Create a C program called `tracer.c`:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/syscall.h>

int main(int argc, char *argv[]){
    printf("Start collecting...\n");
    syscall(471,1);
    sleep(20);
    syscall(471,0);
    printf("Stop collecting...\n");
    return 0;
}
```

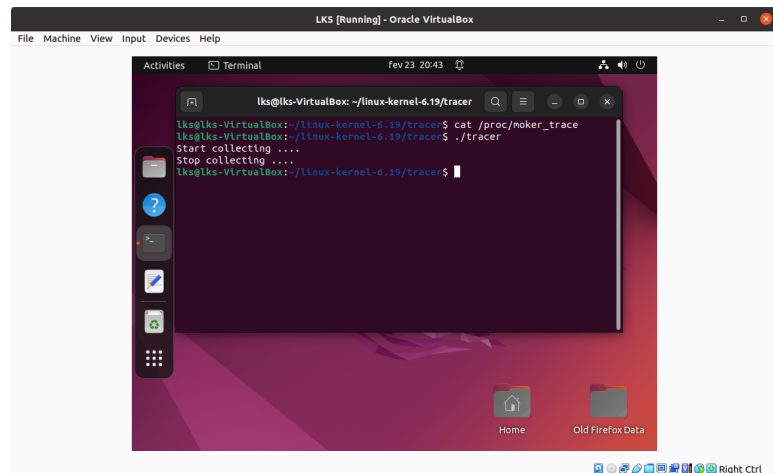
3. Open a terminal and compile it:

```
gcc tracer.c -o tracer
```

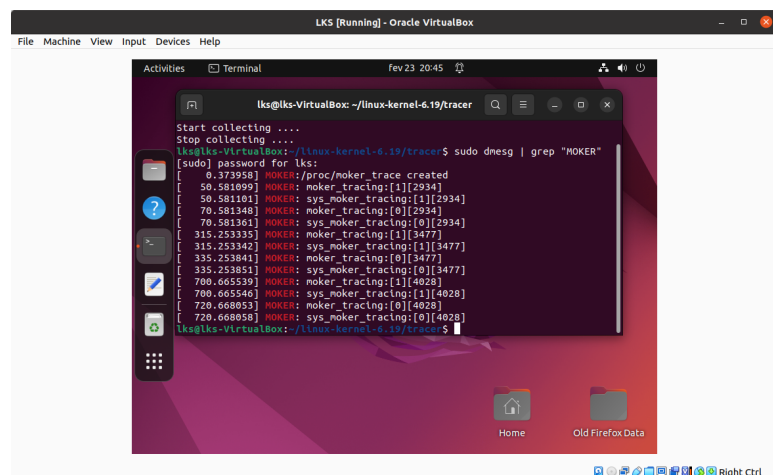
4. Run it

```
./tracer
```

Wait until the program finishing its execution.



`sudo dmesg | grep "MOKER"`



5. Getting the scheduling trace:
`cat /proc/moker_trace`
 In this case there are tracing data.

